

MLOPS ENGINEERING · SESSION 3 OF 5

Orchestration, Inference, Serving & Release

From orchestration to production endpoint — DAGs, inference patterns, serving tools, safe rollout

Airflow · Inference Patterns · Model Serving · BentoML/Triton · vLLM · Release Strategies

Aya · MLOps MENA Community

Session Agenda

5 topics · 3–3.5 hours · from orchestration to production rollout

01	**Orchestration**	Apache Airflow — DAGs, architecture, operators and sensors
02	**Inference Patterns**	Why choosing wrong is costly · Web service · Batch scoring · Streaming
03	**Model Serving**	FastAPI → BentoML → GPU (Triton+TRT) → CPU (ONNX+OpenVINO) → LLM (vLLM)
04	**Load Testing**	Locust — measure FastAPI first, then compare after each serving upgrade
05	**Release Strategies**	How to update your BentoML service safely — Canary, A/B, Blue/Green, Shadow

TOPIC 01

Orchestration

Apache Airflow — DAGs, architecture, operators and sensors

Orchestration — why we need it

A cron job in a file is not a pipeline. A DAG is.

WITHOUT ORCHESTRATION

- Cron jobs in random shell scripts
- No retry on failure — silent data loss
- No visibility into what ran or failed
- Can't define task dependencies
- Re-runs everything even if data didn't change

WITH A DAG ORCHESTRATOR

- Tasks defined with explicit dependencies
- Automatic retry with backoff on failure
- Full UI: status, logs, duration per task
- Trigger: schedule, sensor, or event
- Skip unchanged tasks — incremental runs

DAG = Directed Acyclic Graph

One weekly retraining pipeline, five tasks, one direction of travel

`extract_data` → `validate` → `train_model` → `evaluate` → `register_model`

DIRECTED

- Arrows flow one way — output of one task is input to the next

ACYCLIC

- No loops — task A cannot depend on itself, directly or indirectly

Airflow architecture — five components

Understand what each piece does before writing your first DAG

01

Web Server

Flask-based UI. View DAG graph, trigger runs, inspect task logs, clear failed tasks. Port 8080 by default.

02

Scheduler

Reads DAG files continuously, checks schedules and dependencies, puts tasks on the queue when they're ready to run.

03

Executor

Decides HOW tasks run. SequentialExecutor (dev), LocalExecutor (single machine), CeleryExecutor (distributed), KubernetesExecutor (K8s pods).

04

Worker

The process that actually runs the task code. With CeleryExecutor, workers run on separate machines for parallelism.

05

Metadata DB

PostgreSQL or SQLite. Stores DAG definitions, task states, connections, and variables. The source of truth for everything Airflow knows.

HOW THEY CONNECT

Scheduler queues ready tasks → Worker runs them → Metadata DB stores state → Web Server reads it

Apache Airflow — DAG anatomy

Define pipelines as code; schedule, monitor, and retry automatically

dags/retrain_pipeline.py

```python
# dags/retrain_pipeline.py
from datetime import datetime, timedelta
from airflow import DAG
from airflow.operators.python import PythonOperator

dag = DAG(
    dag_id="ride_duration_retrain",
    schedule="@weekly",          # every Monday 00:00 UTC
    start_date=datetime(2024, 1, 1),
    catchup=False,               # no backfill
    default_args={
        "retries": 2,
        "retry_delay": timedelta(minutes=5),
        "owner": "ml-team",
    },
)
```

dag_id

Unique name for the pipeline. Shows in the Airflow UI. Used in CLI commands and monitoring.

schedule

Cron syntax or shorthand: @daily, @weekly, @hourly. None = manual trigger only.

catchup=False

Don't run missed historical intervals on startup. Almost always what you want for ML pipelines.

Apache Airflow — DAG anatomy

Three callables — extract, train, evaluate

dags/retrain_pipeline.py

```python
def extract(**ctx):
    from google.cloud import storage
    client = storage.Client()
    bucket = client.bucket("mlops-ride-duration")
    bucket.blob("rides/latest.parquet") \
          .download_to_filename("data/raw/rides.parquet")

def train(**ctx):
    import mlflow, subprocess
    with mlflow.start_run() as run:
        subprocess.run(["python", "src/train.py"], check=True)
        mlflow.log_metric("train_mae", train_mae)

def evaluate(**ctx):
    best_run = client.search_runs(experiment_ids=["1"],
                    order_by=["metrics.mae ASC"])[0]
    # compare MAE against current Production model
```

One callable per task

Keep task functions small and idempotent — Airflow may retry any of them.

Imports inside the function

Heavy imports belong inside the callable — the scheduler parses DAG files constantly.

Apache Airflow — DAG anatomy

Three PythonOperators, one dependency chain

dags/retrain_pipeline.py

```
1
2  t_extract  = PythonOperator(task_id="extract",
3                  python_callable=extract,  dag=dag)
4  t_train    = PythonOperator(task_id="train",
5                  python_callable=train,    dag=dag)
6  t_evaluate = PythonOperator(task_id="evaluate",
7                  python_callable=evaluate, dag=dag)
8
9  t_extract >> t_train >> t_evaluate
```

>> operator

Defines task dependencies. t_extract >> t_train means train runs only after extract succeeds.

One callable per task

Keep task functions small and idempotent — Airflow may retry any of them.

Operators — what a task does

The building blocks — every task in a DAG is one of these

PythonOperator

Runs any Python callable. The most common operator in ML pipelines.

```
PythonOperator(task_id='train',
python_callable=train_fn, dag=dag)
```

BashOperator

Runs a bash command. Useful for CLI tools, dvc repro, shell scripts.

```
BashOperator(task_id='dvc', bash_command='dvc
repro', dag=dag)
```

BranchPythonOperator

Returns a task_id to execute next. Enables conditional branching in DAGs.

```
BranchPythonOperator(task_id='check',
python_callable=decide, dag=dag)
```

EmailOperator

Sends an email on completion or failure. Common for alerting on model drift.

```
EmailOperator(task_id='alert', to='team@co.com',
subject='Done', dag=dag)
```

Sensors — wait for a condition

Airflow operators and sensors — a sensor blocks until its condition is true

FileSensor

Waits until a file or folder appears in a local or remote path. Perfect for waiting on upstream data drops.

```
FileSensor(task_id="wait_data",
filepath="gs://bucket/rides/lat
est.parquet", poke_interval=60)
```

ExternalTaskSensor

Waits for a task in another DAG to complete. Coordinates across independent pipelines without coupling them.

```
ExternalTaskSensor(task_id="wai
t_etl",
external_dag_id="etl_pipeline",
external_task_id="load")
```

HttpSensor

Polls an HTTP endpoint until it returns a success response. Use it to wait for a model server to become healthy.

```
HttpSensor(task_id="wait_api",
endpoint="/health",
http_conn_id="api_conn")
```

TOPIC 02

Inference patterns

Web service, batch scoring, streaming — and why the wrong choice is expensive

The most expensive ML mistake

It isn't a bad model — it's deploying a good one the wrong way

01

Cost blowout

Scenario: Team deploys a recommendation model as a REST API (always-on server).

Result: ML server runs 24/7 at $800/month — but recs only need updating nightly. Batch would cost $12/month.

Wrong pattern = 66× more expensive than necessary.

02

Latency SLA breach

Scenario: Team deploys a fraud detection model as a nightly batch job.

Result: Fraudulent transactions process all night, flagged next morning. $2M in fraud slips through before the batch runs.

Wrong pattern = useless model even at 99% accuracy.

03

Over-engineered

Scenario: Team builds a Kafka streaming pipeline for a weekly sales forecast model.

Result: Engineers spend 3 months on streaming infra. A 10-line batch job would have worked identically.

Wrong pattern = wasted 3 months on unnecessary complexity.

Answer before choosing any tool: does my user wait for a prediction? → web service. Can I batch in advance? → batch. Must I react in under a second? → streaming.

Three inference patterns

Same model — three very different serving shapes

01 Web Service

`REST API`

User clicks 'predict' — chatbot, recommendation, ETA. Uber ETA serves 100M+ predictions/day via REST.

Latency	< 200 ms
Throughput	1–1000 req/s
Cost	Medium — server always running
Tool	FastAPI / Litestar (BentoML next)

02 Batch Scoring

`Scheduled Job`

Score all customers nightly. Spotify recomputes recommendations for 600M users every night — no user waits.

Latency	Minutes to hours
Throughput	Millions of rows/job
Cost	Low — compute only when running
Tool	Python + Parquet + Airflow/Ray

03 Streaming

`Event-driven`

React to every event instantly. PayPal scores every transaction in < 200ms — batch scoring would let fraud slip through.

Latency	< 1 second
Throughput	High volume, continuous
Cost	Medium — workers scale to demand
Tool	Kafka / Redis Streams + Python

Batch scoring — score millions on a schedule

No user waiting: read → predict → write results back

src/batch_score.py

```python
# src/batch_score.py
import pandas as pd, mlflow.sklearn
from pathlib import Path
from datetime import date

def load_production_model():
    return mlflow.sklearn.load_model(
        "models:/RideDurationModel/Production")

def batch_score(input_path: str, output_path: str) -> None:
    df = pd.read_parquet(input_path)
    print(f"Scoring {len(df):,} rows …")
    model    = load_production_model()
    features = df[["distance_km", "passengers",
                   "hour_of_day"]].values
    df["pred"] = model.predict(features)
```

Load from Registry

Always load from the Production stage in the MLflow Registry — not a hardcoded path. New version promoted → code picks it up automatically.

Parquet over CSV

10–100× faster for large datasets. Columnar — only reads the columns needed. Standard in production data pipelines.

No latency SLA

Batch jobs run for minutes or hours. No user waits. Optimize for throughput and cost, not latency.

Batch scoring — write and schedule

Stamp the run, write the Parquet, wrap it in the DAG you already built

src/batch_score.py

```
    df["run_date"] = date.today().isoformat()
    Path(output_path).parent.mkdir(parents=True,
                                   exist_ok=True)
    df.to_parquet(output_path, index=False)
    print(f"Done → {output_path}")

if __name__ == "__main__":
    batch_score(
        input_path  =
f"data/scoring/input/{date.today()}.parquet",
        output_path =
f"data/scoring/output/{date.today()}.parquet",
    )
```

run_date column

Always stamp predictions with their generation date. Critical for debugging drift — which batch produced the anomalous results?

Schedule with Airflow

Use a PythonOperator to wrap batch_score() in the DAG you built in Topic 01 — same function, runs on the weekly schedule automatically.

Streaming inference — event-driven

Predict on every event as it arrives, at any scale

KAFKA PATH — HIGH-THROUGHPUT, REPLAY-ABLE

App / IoT
Producer
→
Kafka
Topic
→
Kafka
Consumer
→
ML Model
loaded once
→
PostgreSQL
/ Redis

REDIS STREAMS PATH — LOW-LATENCY, SIMPLER SETUP

App / IoT
Producer
→
Redis
Stream
→
Redis
Consumer
→
ML Model
loaded once
→
Redis Hash
/ PostgreSQL

Both paths: same ML model code, same consumer.py. Only the broker client library changes — kafka-python vs redis. The model never knows which broker it is behind.

What is a message broker?

A middle layer that receives events from producers and delivers them to consumers — the app doesn't need to know the model exists

Apache Kafka

Distributed log. High-throughput, replay-able, fault-tolerant. Industry standard for high-volume event streams.

Used by: LinkedIn, Uber, Airbnb — millions of events/sec

Redis Streams

In-memory stream built into Redis. Low-latency, simpler setup than Kafka. Perfect for small-to-medium throughput.

Used by: Real-time dashboards, session events, IoT — < 100k events/sec

RabbitMQ

Traditional message queue. Task-oriented — delivers one message to one consumer. Simpler mental model than Kafka.

Used by: Job queues, delayed tasks, microservice communication

For a first project: use Redis Streams. For production at scale: use Kafka. The consumer code is identical either way.

Streaming — consumer code

A long-running process that calls the model on every incoming message

consumer.py

```python
# consumer.py — event-driven inference consumer
import json, mlflow.sklearn, os

MODEL = None  # loaded once on cold start

def get_model():
    global MODEL
    if MODEL is None:
        MODEL = mlflow.sklearn.load_model(
            os.environ["MODEL_URI"])
    return MODEL
```

MODEL = None

Loaded once per container lifetime, cached in memory. Every event after the first uses the cached model — no disk read.

Scale out

Run multiple consumer containers in parallel to increase throughput. Kafka and Redis distribute messages across them automatically.

Streaming — one function per event

And a container that never stops

consumer.py

```python
def predict_on_event(event: dict) -> dict:
    """Called for every message from the broker."""
    model   = get_model()
    payload = json.loads(event["data"])
    features = [payload["distance_km"],
                payload["passengers"],
                payload["hour_of_day"]]
    prediction = model.predict([features])[0]
    result = {"ride_id": payload["ride_id"],
              "prediction": round(float(prediction), 2)}
    store_result(result)   # PostgreSQL, Redis, file
    return result

# Run as a long-running consumer process:
#   python consumer.py
# or wrap in Docker with --restart unless-stopped
```

Same code, any broker

predict_on_event() doesn't care whether the message came from Kafka, Redis Streams or RabbitMQ. The broker integration is a thin wrapper around it.

store_result()

Write to PostgreSQL, Redis, a Parquet file, or push to a downstream topic. The model is decoupled from the storage choice.

--restart unless-stopped

Docker restart policy. If the consumer crashes or the machine reboots, Docker restarts it automatically.

Streaming vs batch — how to choose

Streaming is powerful but overkill in most ML scenarios

USE STREAMING WHEN…

- Decisions must be made in < 1 second after the event (fraud, anomaly)
- Event volume is continuous and unbounded — not time-bounded
- Data loses value if batched even for minutes (trading, IoT safety)
- Each event is independent and stateless — no joins across events
- Team has capacity to run and monitor an always-on consumer process

STICK WITH BATCH WHEN…

- Results can be pre-computed daily or hourly (recommendations, credit scores)
- Data requires aggregation across many events (weekly totals, rolling averages)
- Model inference is slow (> 500ms) — batching amortizes the cost
- Team is small and streaming infra adds operational complexity
- You're not sure yet — start with batch, migrate to stream if needed

If your stakeholder says 'nightly is fine' — use batch. Streaming is not inherently better, just more expensive to build and operate.

TOPIC 03

Model serving

What it is · why it's hard · the vocabulary · the stack · the tools

Topic 02 answered: how should I deploy? Topic 03 answers: how do I build a serving system that actually works under load?

Note: the code examples in this topic use image classification (ResNet-50) instead of ride duration — GPU concepts are clearer with a PyTorch vision model. The principles apply to any model.

TOPIC 03 · MODEL SERVING DEFINITION

mod·el serv·ing

/ˈmädl sərviNG/ · noun

The process of converting a machine learning model into an API or service that can be distributed to and consumed by individuals, teams, IT environments, or software applications in a highly scalable production environment.

01

Convert a machine learning model into an API

02

Distribute (or deploy) the API so it can be consumed by other people or apps

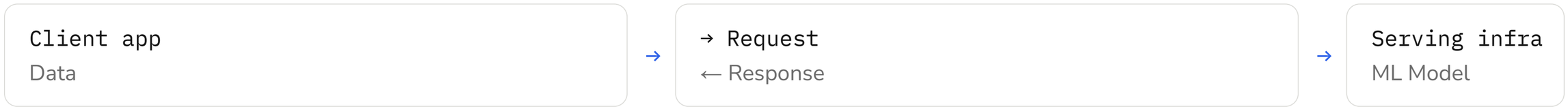
03

Host the deployed service on enterprise infrastructure to handle large scale

Source: Modzy — What is Model Serving?

What is model serving?

Making a trained model available to the world — reliably, at scale, with the ability to update and monitor



SERVING = INFERENCE + EVERYTHING AROUND IT

01

HTTP/gRPC endpoint

Converts your model into something other systems can call

02

Reliability and scale

Handles concurrent users, retries, health checks, versioning

03

Operations

Monitor, update, roll back — without restarting or rewriting

Source: TensorFlow Serving overview

"Congratulations, your model is useless."

THE STORY

A team spent 3 months on ResNet-50 image classification. They cleaned data, tried 5 architectures, tuned the learning rate, and finally hit 94% top-1 accuracy. They saved model.pth. Everyone celebrated.

Then the product team asked:

"Great. Where do I send the image?"

And the room went quiet.

A model checkpoint is not a product. It's a file.

The gap

Training is a research problem. Serving is a systems problem.

```
model.pth
```

→ A URL that returns a prediction in 40ms under 500 req/s

TRAINING

- A research problem — accuracy, architecture, data quality

SERVING

- A systems problem — latency, throughput, concurrency, cost

They fail for completely different reasons.

Inference vs serving

Two words everyone uses interchangeably. They are not the same.

Inference

Running a forward pass. One input, one output. model(x). A mathematical operation.

Serving

Making inference available reliably — at required latency, throughput and cost — with the ability to update, monitor and roll back.

Inference is a function call. Serving is everything you build around that function call so other people can depend on it.

The vocabulary that matters

Six terms that decide every serving argument you will have

TERM	DEFINITION	WHY IT MATTERS
Latency	Time from request sent to response received	What the user feels
p50 / p95 / p99	Percentiles of the latency distribution	The average lies. p99 is what breaks your SLA
Throughput	Requests served per second	What determines your infrastructure bill
Batch size	Inputs processed in one forward pass	The main lever between latency and throughput
GPU utilization	% of GPU compute actually doing work	Naive serving: 8–15%. That gap is why this field exists.
Cold start	Time from container start to first prediction	Kills serverless and autoscaling designs

Ask the room: what GPU utilization does a naive Flask + ResNet endpoint get under real traffic? Most guess 60–80%. Answer: 8–15%.

The four-layer serving stack

Every tool in this topic owns some subset of these layers

1	**Transport & API Layer**	HTTP/gRPC endpoint · Auth & validation · Request schema · Response schema · OpenAPI docs
2	**Service & Scheduling Layer**	Dynamic batching · Request queueing · Pre/post-processing · Model versioning · Concurrency control
3	**Inference Runtime**	The engine that actually runs the graph: eager PyTorch / ONNX Runtime / TensorRT / OpenVINO
4	**Hardware**	GPU (NVIDIA A100/H100) · Intel CPU + AVX-512 · Apple ANE · Edge accelerators (Jetson)

Tool ownership at a glance

Compare by layers owned, not by feature lists

FastAPI	BentoML	Triton	KServe	vLLM
L1 only	L1 + L2	L1 + L2 + L3	L1 + L2 + K8s	L1 + L2 + L3 · LLM only

FastAPI, BentoML and Triton are not three answers to the same question. Comparing them directly is like comparing a steering wheel, a gearbox and an engine.

The serving progression

Each step solves a problem the previous step left open

CATEGORY	TOOLS	THROUGHPUT	GPU UTIL	CHOOSE WHEN
CAT 1 Native Python	FastAPI / Flask	~10 req/s	8–15%	Prototype, internal tool, < 50 req/s
CAT 2 ML Framework	BentoML	~200 req/s	40–55%	Most production ML teams. Best starting point.
CAT 3 GPU-Accelerated	TensorRT + Triton	~2000 req/s	80–92%	GPU cost is binding. p99 latency is binding.
CAT 4 CPU-Accelerated	ONNX Runtime + OpenVINO	~500 req/s on CPU	N/A — no GPU	No GPU. Cloud cost-optimized. Edge deployment.
CAT 5 LLM Serving	vLLM / SGLang / TGI	KV-cache aware	85–95%	Generative models only. Different bottleneck entirely.

Start at CAT 2 (BentoML). Move right only when you have a measured reason: a Locust report showing GPU utilization below 50%, or p99 above SLA.

A decision tree, not a checklist

Move right ONLY when you have a measured reason

CAT 1

FastAPI

Move right if: GPU < 15%? p99 > 4s at 100 users?

CAT 2

BentoML

Move right if: GPU < 55%? p99 still > SLA?

CAT 3

TRT + Triton

Move right if: GPU < 90%? Still not fast enough?

CAT 4

ONNX / OpenVINO

Move right if: No GPU? Need CPU-only?

CAT 5

vLLM

Move right if: Generative LLM?

01

Run Locust BEFORE choosing a tool

If you haven't measured, you don't know what to fix.

02

Most teams stop at CAT 2

BentoML solves 90% of production serving problems. CAT 3+ adds real DevOps cost.

03

Never skip to CAT 3 without CAT 2 data

'Triton will be faster' is not a reason. A Locust report is a reason.

Measure with Locust first, at every step. Curiosity is not a measured reason.

FastAPI — the first working endpoint

Owns exactly one layer. Everything else is your problem.

main.py

```python
# Level 0 — model.pt on disk, callable by nobody
model = torchvision.models.resnet50(
            weights="DEFAULT").eval().cuda()
out = model(preprocess(Image.open("cat.jpg"))
              .unsqueeze(0))    # works on your laptop

# Level 1 — FastAPI: give it a URL
from fastapi import FastAPI, UploadFile
import torchvision, torch, io

app   = FastAPI()
model = torchvision.models.resnet50(
            weights="DEFAULT").eval().cuda()

transform = torchvision.transforms.Compose([
    torchvision.transforms.Resize(256),
    torchvision.transforms.CenterCrop(224),
    torchvision.transforms.ToTensor(),
])
```

Level 0

A checkpoint on disk. It works on your laptop and nowhere else.

Level 1

FastAPI gives the model an address. This is the smallest useful step.

Layers owned

Layer 1 only — HTTP transport. Batching, concurrency and runtime are still your problem.

The endpoint, and what it buys you

Genuine progress — now run 100 concurrent clients and watch nvidia-smi

main.py

```python
@app.post("/predict")
async def predict(file: UploadFile):
    img = transform(Image.open(
        io.BytesIO(await file.read()))).unsqueeze(0).cuda()
    with torch.inference_mode():
        logits = model(img)
    return {"class_id": int(logits.argmax()),
            "score": float(logits.max())}

# uvicorn main:app --host 0.0.0.0 --port 8000
# You now have: a URL, Pydantic validation,
# and auto-generated OpenAPI docs.
```

A URL

Any client on the network can call /predict. A genuine milestone.

Validation

Pydantic catches bad input before it reaches the model.

OpenAPI docs

Auto-generated /docs — testable in a browser immediately.

Layers owned

Layer 1 only. HTTP transport is solved; nothing else is.

FastAPI under load — four problems

Every problem here has a name, a cause, and a tool that fixes it

BENCHMARK — SAME RESNET-50, SAME GPU (A100)

1 user → 85ms p95 · 100 concurrent users → 4,200ms p95 · GPU utilization: 9%

01 Batch size is always 1

GPU is designed to process 32–64 images per forward pass. FastAPI sends one image per request. You're paying for a GPU and using ~9% of it.

Fix: Dynamic batching — collect concurrent requests, run as one batch, fan results back.

Layer 2 — BentoML / Triton

02 Python GIL blocks concurrency

async def predict runs on Python's event loop, but model(img) is a blocking PyTorch call. It holds the GIL, freezing all other coroutines while the model runs.

Fix: Run the model in a separate process (BentoML Runner) so the HTTP server stays truly async.

Layer 2 — BentoML Runner

FastAPI under load — four problems

The runtime gap and the operations gap

03 Eager PyTorch is slow

PyTorch eager mode executes op-by-op through the Python interpreter. No graph fusion, no memory planning, no kernel selection. Conv+BN+ReLU = 3 separate kernel launches.

Fix: Compile the graph. ONNX → TensorRT fuses them into one kernel and plans memory.

`Layer 3 — TensorRT / ONNX Runtime`

04 No model versioning or health checks

Deploying v2 = restart the server. Rolling back = redeploy. Canary test = maintain two separate services. No /health endpoint for load balancers.

Fix: BentoML save_model() gives you versioned artifacts plus built-in /healthz and /readyz.

`Layer 1+2 — BentoML`

TOPIC 04

Load testing

Locust — measure before you optimize anything

Load testing — what it is, why ML APIs need it

You don't know if your service is production-ready until it breaks under load

WHAT IS LOAD TESTING?

- Simulating many concurrent users hitting your API simultaneously
- The maximum requests/second before failures appear
- Latency percentiles (p50/p95/p99) under real load
- The exact component that breaks first

WHY ML APIS SPECIFICALLY?

- Model inference is much slower than a DB query
- Python GIL causes thread contention under concurrency
- Model loading on cold start adds unpredictable latency spikes
- Memory grows silently until an OOM crash at N users
- One blocking predict() starves all other requests

Also critical for LLMOps

Streaming tokens changes everything — four LLM-specific dimensions

01

TTFT

Time to First Token — how long until the first token streams back. The UX metric users feel.

02

TBT

Time Between Tokens — token generation speed. Should be < 50ms per token for smooth streaming.

03

Concurrency

vLLM uses continuous batching. Load testing reveals the optimal batch size at your SLA.

04

Context length

Long prompts = more memory = fewer concurrent requests. Load test at your p95 context length.

RPS
Throughput

p50
Median

p95
What most users care about

p99
Tail / outliers

0%
Errors — should be zero

Discord: an ML message-classification API broke at 200 concurrent users due to GIL contention on the model thread. Locust caught it before launch — the fix was an async BentoML runner.

Load testing tools — Locust and alternatives

Five tools, one recommendation for ML teams

TOOL	ECOSYSTEM	WHAT IT GIVES YOU	BEST FOR
Locust	★ Recommended · Python	Write scenarios in pure Python · Web UI plus headless/CSV mode · Easy Docker/K8s deployment · Native async support	ML and LLMOps teams — same language as your model code
k6	JS / Go	Grafana-native dashboards · Scripted in JavaScript (ES6) · Very fast — compiled Go runtime · Cloud execution via Grafana k6 Cloud	Frontend and full-stack teams already in the JS ecosystem
Artillery	Node.js	YAML-first — minimal code for simple tests · Plugin ecosystem · Good for REST and WebSocket · AWS Lambda execution mode	Quick HTTP tests without writing Python or JS classes
Gatling	JVM	Scala DSL, very high scale, detailed HTML reports	Java/Scala teams, enterprise, very high scale
Apache JMeter	Classic	GUI-driven, protocol diversity, long history	QA teams, legacy environments, protocol diversity

For ML and LLMOps: start with Locust. Switch to k6 if you need Grafana dashboards or higher virtual user counts from one machine.

Locust — writing tasks

One class, one simulated user, realistic payloads

locustfile.py

```python
# locustfile.py
from locust import HttpUser, task, between
import random

class MLAPIUser(HttpUser):
    wait_time = between(0.5, 2.0)

    @task(weight=10)
    def predict(self):
        payload = {
            "distance_km": round(
                random.uniform(0.5, 30.0), 2),
            "passengers":  random.randint(1, 4),
            "hour_of_day": random.randint(0, 23),
        }
```

HttpUser

One instance = one simulated user. Locust spawns N instances, each running tasks independently.

wait_time

Random pause between tasks. between(0.5, 2.0) simulates realistic human behaviour.

@task(weight=10)

Ten times more likely than a weight-1 task. Mirrors real traffic patterns — predict far outnumbers health.

Locust — reading results

Assert on the response, then run it headless in CI

locustfile.py

```
        with self.client.post("/predict", json=payload,
                catch_response=True) as resp:
            if resp.status_code != 201:
                resp.failure(f"Expected 201, got"
                             f" {resp.status_code}")
            elif resp.json().get("duration_min", -1) < 0:
                resp.failure("Negative duration")

    @task(weight=1)
    def health(self):
        self.client.get("/health")

# locust -f locustfile.py --host http://localhost:8000
#        --users 100 --spawn-rate 10
#        --run-time 2m --headless --csv=results/load
```

catch_response

Manually mark a response as a failure even on HTTP 200 — useful when the API returns errors inside 200 responses.

--headless + CSV

CI-friendly: export load_test_stats.csv and fail the build if p95 exceeds your threshold.

BentoML — fixes all four FastAPI problems

One framework, layers 1 + 2. Still delegates the runtime to PyTorch.

PROBLEM	FASTAPI ALONE	BENTOML
Batch size	Always 1 — GPU at 9%	batchable=True → micro-batching → GPU at 45–55%
GIL blocking	model() blocks the event loop	Runner = separate process. HTTP stays async.
Model versioning	Restart to deploy v2	bentoml save_model() — versioned, immutable tags
Health checks	You write the endpoint	Built-in /healthz and /readyz
Packaging	Dockerfile written by hand	bentoml containerize → Docker image in one command
Layers owned	Layer 1	Layers 1 + 2

AFTER BENTOML

- 100 users → 380ms p95 · GPU: 45–55% · Throughput: ~200 req/s

STILL MISSING

- Runtime is still PyTorch eager FP32. No graph fusion, no INT8. GPU at 55%, not 90% → next: layer 3.

BentoML — service definition and micro-batching

Save the model, define the Runner, expose an async endpoint

service.py

```python
import bentoml, numpy as np, mlflow.sklearn
from pydantic import BaseModel

# Step 1: Save model to the BentoML store
sk_model = mlflow.sklearn.load_model(
    "models:/RideDurationModel/Production")

bento_model = bentoml.sklearn.save_model(
    "ride_duration", sk_model,
    signatures={"predict": {"batchable": True,
                            "batch_dim": 0}},
)
```

batchable=True

One flag enables micro-batching. BentoML collects concurrent requests into a batch and sends them together to the Runner.

bentoml containerize

Builds a self-contained Docker image with model weights, code and dependencies baked in. No manual Dockerfile.

BentoML — the Runner and the async endpoint

The Runner owns the model; the HTTP loop never blocks

service.py

```python
# Step 2: Define the Service
class PredictRequest(BaseModel):
    distance_km: float; passengers: int; hour_of_day: int

class PredictResponse(BaseModel):
    duration_min: float

runner = bentoml.sklearn.get(
    "ride_duration:latest").to_runner()
# Runner = separate process that owns the model

svc = bentoml.Service("ride_api", runners=[runner])

@svc.api(input=bentoml.io.JSON(pydantic_model=PredictRequest),
output=bentoml.io.JSON(pydantic_model=PredictResponse))
async def predict(req: PredictRequest) -> PredictResponse:
    features = np.array([[req.distance_km,
        req.passengers, req.hour_of_day]])
    result = await runner.predict.async_run(features)
    return PredictResponse(duration_min=float(result[0]))
```

to_runner()

Spawns a dedicated process for the model. The HTTP event loop is never blocked by model execution. The GIL problem is solved.

async_run()

A non-blocking call to the Runner. Other requests execute while waiting, and BentoML batches them automatically.

Still PyTorch eager

The Runner still uses PyTorch FP32. To fix the runtime, swap the backend to ONNX Runtime or point it at Triton.

CAT 3

GPU acceleration

Graph optimization, reduced precision, device concurrency

CAT 3 · GPU ACCELERATION

GPU acceleration — three independent levers

Each lever is multiplicative with the batching gain from BentoML

A

Graph Optimization

Compile the model to an optimized graph. Conv+BN+ReLU fuse into one kernel. Constants fold. Memory layout planned ahead of time.

```
torch.compile · TorchScript · ONNX export · TensorRT engine build
```

B

Reduced Precision

FP32 → FP16 → INT8. Post-training INT8 with ~300 calibration images costs < 0.5% top-1 accuracy on ResNet. INT4 for LLMs.

```
TensorRT INT8 calibration · ONNX Runtime quantize_static · QAT
```

C

Device Concurrency

Run 2–4 model instances on one GPU so compute overlaps with host↔device memory transfers. The GPU never sits idle between requests.

```
Triton model_instance_group · BentoML runner concurrency · Ray Serve replicas
```

RESNET-50 ON A100 — SAME HARDWARE

1×
PyTorch eager FP32 — baseline

~2×
+ ONNX Runtime

~4×
+ TensorRT FP16

7–10×
+ TensorRT INT8

NVIDIA Triton — owns layers 1+2+3

The production GPU serving standard: dynamic batching + TensorRT + multi-model

export_onnx.py

```python
# 1. Export the model to ONNX (or a TensorRT engine)
import torch, torchvision

model = torchvision.models.resnet50(
            weights="DEFAULT").eval().cuda()
dummy = torch.randn(1, 3, 224, 224, device="cuda")

torch.onnx.export(model, dummy, "model.onnx",
    input_names=["input"], output_names=["output"],
    dynamic_axes={"input": {0: "batch"},
                  "output": {0: "batch"}})

# 2. Triton model repository structure
# model_repo/
#   resnet50/
#     config.pbtxt      <- Triton config
#     1/model.onnx      <- versioned artifact
```

Dynamic axes

Mark the batch dimension as dynamic or Triton cannot batch requests together.

Repository layout

Triton discovers models by folder structure. The numbered folder is the model version.

The result

38ms p95 · 88% GPU utilization · ~2000 req/s — roughly 50× the FastAPI baseline.

config.pbtxt — where the magic happens

Batching and GPU concurrency become configuration, not code

config.pbtxt

```
name: "resnet50"
backend: "onnxruntime"      # or "tensorrt"
max_batch_size: 64

dynamic_batching {
  preferred_batch_size: [8, 16, 32]
  max_queue_delay_microseconds: 5000
}

instance_group [{ count: 2, kind: KIND_GPU }]

# 4. Start Triton
# docker run --gpus all -p 8000:8000 \
#   -v ./model_repo:/models \
#   nvcr.io/nvidia/tritonserver:24.01-py3
```

dynamic_batching

config.pbtxt replaces code. max_queue_delay_microseconds: 5000 waits up to 5ms to fill a batch. No Python needed.

instance_group count: 2

Two copies of the model on the same GPU. While one batch runs, the next loads. The GPU sits idle less than 5% of the time.

When to use Triton

GPU cost or p99 is a binding constraint. Multi-framework fleet. Model ensembles.

BentoML + Triton — complements, not competitors

BentoML owns the HTTP layer. Triton owns the GPU layer.

service_triton.py

```python
# A BentoML Runner backed by Triton, not PyTorch
import bentoml
import tritonclient.http as httpclient

class TritonRunner(bentoml.Runner):
    def __init__(self):
        self.client = httpclient.InferenceServerClient(
            "localhost:8000")
        super().__init__(name="triton_resnet50")

    def run(self, inputs: np.ndarray) -> np.ndarray:
        inp = httpclient.InferInput("input",
                  inputs.shape, "FP32")
        inp.set_data_from_numpy(inputs)
        result = self.client.infer("resnet50", [inp])
        return result.as_numpy("output")

runner = TritonRunner()
svc = bentoml.Service("resnet_prod", runners=[runner])
```

The Runner is a client

It forwards tensors to Triton over HTTP or gRPC instead of calling PyTorch in-process.

Who owns what

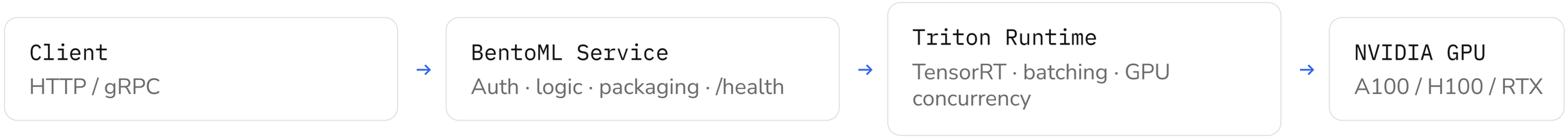
BentoML: HTTP, auth, validation, packaging. Triton: batching, runtime, GPU concurrency.

One deployment

Both run as containers in the same compose file or Kubernetes namespace.

The production stack

Who owns what, from the client down to the GPU



BENTOML STILL OWNS

- HTTP endpoint · Input validation · Business logic
- Auth and rate limiting · Docker packaging · Health checks

TRITON NOW OWNS

- Dynamic batching · TensorRT runtime · GPU concurrency
- Multi-model serving · gRPC + HTTP on GPU · Model versioning (GPU side)

"Which one?" is the wrong question. "Which layer is my bottleneck?" is the right one.

CAT 4

CPU-accelerated serving

ONNX Runtime and OpenVINO — no GPU needed

CPU-accelerated serving — ONNX Runtime

No GPU needed. 3–8× faster than PyTorch eager on CPU.

cpu_serving.py

```python
# Option A: ONNX Runtime — works everywhere
from onnxruntime.quantization import (
    quantize_static, CalibrationDataReader, QuantType)
import onnxruntime as ort, numpy as np

# 1. Export the PyTorch model to ONNX
torch.onnx.export(model.cpu(), dummy.cpu(), "model.onnx",
    dynamic_axes={"input": {0: "batch"}}, opset_version=17)

# 2. Quantize to INT8 (~300 calibration samples)
class MyCalibReader(CalibrationDataReader):
    def __init__(self, X):
        self.data = iter([{"input": X[i:i+8].numpy()}
                          for i in range(0, len(X), 8)])
    def get_next(self): return next(self.data, None)

quantize_static("model.onnx", "model_int8.onnx",
    CalibrationDataReader=MyCalibReader(X_calib),
    quant_format=QuantType.QInt8)
```

When to choose CPU

No GPU available · cloud cost-optimized fleet (CPU VMs 10× cheaper) · edge deployment · data can't leave a CPU-only environment.

quantize_static

Post-training INT8 needs ~300 representative samples. Accuracy cost is typically under 1%.

Cross-platform

ONNX Runtime runs on any x86, ARM or cloud VM. The best default for CPU serving.

Option B: OpenVINO on Intel CPU

Same model, four runtimes — pick on hardware, not preference

openvino_serving.py

```python
# Option B: OpenVINO (Intel CPU)
from openvino.runtime import Core
from openvino.tools.mo import convert_model

ov_model  = convert_model("model.onnx")
core      = Core()
compiled  = core.compile_model(ov_model, "CPU")
infer_req = compiled.create_infer_request()

output = infer_req.infer(
    {"input": X.numpy()})["output"]
# 3-5× faster than ONNX Runtime on Intel

# 3. Or just run ONNX Runtime directly:
sess = ort.InferenceSession("model_int8.onnx",
    providers=["CPUExecutionProvider"])
out  = sess.run(None, {"input": X.numpy()})[0]
```

PyTorch eager (CPU)

Baseline. Not recommended for production CPU serving.

ONNX Runtime

~2–3×. Cross-platform. The best default for CPU serving.

ONNX Runtime INT8

~4–6×. Under 1% accuracy cost. Best for throughput-sensitive CPU deployments.

OpenVINO INT8

~5–8×. Intel hardware only, AVX-512 SIMD. Best absolute CPU performance.

Why LLMs need a completely different stack

Everything in CAT 1–4 is correct for ResNet and wrong for a 70B LLM

DIMENSION	CLASSICAL MODELS	LLMS (LLAMA, GPT, MISTRAL)
Compute shape	Fixed — one forward pass	Autoregressive — N forward passes per response, one per token
Batching	Dynamic batching of fixed-size inputs	Continuous / in-flight — new requests join mid-generation
Memory concern	Weights + activations (manageable)	KV cache — grows with context length, fragments GPU memory
Key metric	p95 latency	TTFT (Time to First Token) · TBT (Time Between Tokens)
Main optimization	TensorRT INT8 · DALI preprocessing	PagedAttention · prefix caching · speculative decoding
Production stack	Triton + TensorRT / BentoML	vLLM · SGLang · TensorRT-LLM · TGI
GPU waste cause	batch_size too small → low compute	KV cache fragmentation → 80% memory wasted without PagedAttention

Don't use Triton for a 70B chat model. Don't use vLLM for ResNet. The tooling is specialized for a reason.

CAT 5 · VLLM CONCEPT

vLLM — production LLM serving at scale

PagedAttention turns GPU memory management upside down

THE CORE PROBLEM

The KV cache must be pre-allocated per request. With 10 concurrent users, GPU memory fragments — up to 80% wasted — and the server handles far fewer users than the hardware allows.

WITHOUT PAGEDATTENTION

- KV cache pre-allocated per request
- 80% of GPU memory wasted

WITH PAGEDATTENTION

- KV cache in non-contiguous pages, like OS virtual memory
- 3–24× more throughput

5 → 120
req/s on Mistral 7B

1×
same A100 GPU

24×
throughput gain

What vLLM brings

Six capabilities you would otherwise build yourself

01

Continuous Batching

New requests join mid-generation. GPU stays at ~95% utilization.

02

OpenAI-Compatible API

Drop-in replacement — change base_url only. Zero refactoring.

03

Streaming Tokens

SSE streaming native. Users see tokens as they generate.

04

50+ Models

Llama, Mistral, Qwen, Gemma, Phi — HuggingFace or local.

05

Quantization Native

AWQ, GPTQ, INT8 natively. Run 70B on one A100 with AWQ-4bit.

06

Tensor Parallelism

--tensor-parallel-size N splits the model across GPUs automatically.

vLLM — one command to serve

OpenAI-compatible: start the server, then call it with the OpenAI SDK

vllm_client.py

```python
# pip install vllm

# 1. Start an OpenAI-compatible server
# vllm serve Qwen/Qwen2.5-7B-Instruct \
#   --host 0.0.0.0 --port 8000 \
#   --tensor-parallel-size 1 \
#   --max-model-len 8192

# 2. Call it — identical to calling OpenAI
from openai import OpenAI

client = OpenAI(base_url="http://localhost:8000/v1",
                api_key="EMPTY")

response = client.chat.completions.create(
    model="Qwen/Qwen2.5-7B-Instruct",
    messages=[{"role": "user",
               "content": "Explain PagedAttention."}],
    max_tokens=256, temperature=0.7,
)
```

One command to serve

vllm serve <model> starts an OpenAI-compatible server. No Flask, no FastAPI, no application code.

OpenAI drop-in

Change base_url to your vLLM server and set api_key='EMPTY'. Existing code works against any local model.

vLLM vs Triton

Triton: classical models. vLLM: generative LLMs only. Not interchangeable.

Streaming is the UX

SSE streaming natively, plus an in-process Python API

vllm_client.py

```python
# 3. Streaming — users see tokens as they generate
stream = client.chat.completions.create(
    model="Qwen/Qwen2.5-7B-Instruct",
    messages=[{"role": "user",
               "content": "Write a haiku about MLOps."}],
    stream=True,
)
for chunk in stream:
    if chunk.choices[0].delta.content:
        print(chunk.choices[0].delta.content,
              end="", flush=True)

# 4. Python API (in-process, no HTTP)
from vllm import LLM, SamplingParams

llm = LLM(model="Qwen/Qwen2.5-7B-Instruct",
          dtype="float16")
params = SamplingParams(temperature=0.8, max_tokens=512)
out = llm.generate(["Tell me about MLOps."], params)
```

Streaming is the UX

Users see the first token in ~200ms instead of waiting 10s for a full response. vLLM streams SSE natively.

BentoML + vLLM

Use BentoML for auth, routing and packaging around vLLM. BentoML owns layer 1; vLLM owns layers 2 and 3.

The serving decision guide

Start at BentoML. Measure before you move right.

CATEGORY	TOOL	P95	GPU	CMPLX	MOVE HERE WHEN…
CAT 1 — Native	FastAPI	~4000ms*	9%	Low	Prototyping or < 10 req/s internal tool
CAT 2 — ML Framework	BentoML	~380ms	50%	Low	Default start for any production ML service
CAT 3 — GPU Accel	BentoML + TRT/Triton	~38ms	88%	Medium	GPU cost binding. p99 > SLA. Measured.
CAT 4 — CPU Accel	ONNX Runtime / OpenVINO	~60ms CPU	N/A	Low–Med	No GPU. Cloud cost-optimized. Edge.
CAT 5 — LLM	vLLM	TTFT ~200ms	92%	Low	Any generative LLM. No exceptions.

** 100 concurrent users, batch_size=1. The rule: run Locust at your target load, check nvidia-smi. GPU below 50% → BentoML batching first. Still slow → TRT/Triton. LLM → vLLM, always. No GPU → ONNX Runtime.*

Cloud managed model serving

Every cloud maps to the same four-layer stack — they just handle different layers for you

AWS Amazon Web Services

CLASSIC ML

SageMaker Real-Time Endpoints · SageMaker Triton (managed)

LLM

Amazon Bedrock (Claude, Llama, Mistral) · SageMaker JumpStart

Most mature. Widest model library. SageMaker Triton = Triton without the DevOps overhead.

GCP Google Cloud

CLASSIC ML

Vertex AI Prediction (custom containers) · Vertex AI Model Garden

LLM

Vertex AI Gemini API · Model Garden (Llama, Mistral on GCP)

Best for TensorFlow / JAX models. AutoML integration. Cheapest GPU spot pricing.

Azure Microsoft Azure

CLASSIC ML

Azure ML Online Endpoints · Azure ML Batch Endpoints

LLM

Azure OpenAI Service (GPT-4, o1) · Azure AI Studio (fine-tune + deploy)

Best OpenAI model access. Enterprise compliance (HIPAA, SOC2). Strongest in regulated industries.

NVIDIA and Hugging Face

Fastest paths from an open model to a GPU endpoint

NVIDIA NVIDIA Cloud / NGC

CLASSIC ML

Triton on BentoCloud · NGC Catalog (optimized containers)

LLM

NIM — NVIDIA Inference Microservices · NIM for LLMs (Llama, Mistral, Nemotron)

Best raw GPU performance. NIM = vLLM + TensorRT-LLM packaged as one-command containers.

HF Hugging Face

CLASSIC ML

Inference Endpoints (custom models) · Inference API (hosted models)

LLM

Text Generation Inference (TGI) · Inference Endpoints for LLMs

Fastest path from an open-source model to an endpoint. TGI is a production-grade vLLM alternative.

All of these abstract away DevOps overhead. You still need the four-layer stack to debug them when they slow down.

Managed cloud vs self-hosted

A real engineering and business decision, not a preference

DIMENSION	SELF-HOSTED (BENTOML / TRITON)	MANAGED CLOUD (SAGEMAKER / VERTEX)
Setup time	Hours to days	Minutes (wizard-driven)
Cost at low traffic	Low — pay for your server only	High — minimum instance costs apply
Cost at high traffic	Low — you control the infra	Can be very high — per-prediction pricing
GPU optimization	Full control (TRT, quantization)	Limited — the provider's runtime
Data privacy	Full control — data never leaves	Verify compliance per provider

Operations, lock-in and compliance

The five dimensions that surface after launch

DIMENSION	SELF-HOSTED (BENTOML / TRITON)	MANAGED CLOUD (SAGEMAKER / VERTEX)
Vendor lock-in	Zero — portable containers	High — provider-specific APIs
Autoscaling	You configure it (K8s HPA, Ray)	Built-in, one checkbox
Model update	A CI/CD pipeline you build	Click-button or SDK call
Debugging	Full access to logs and metrics	Limited — the provider's observability
Compliance (HIPAA etc.)	You certify everything	The provider may be pre-certified

Choose based on where you are

Five situations, five answers

Prototype / MVP	→ Managed cloud	Fastest to endpoint. Don't optimize prematurely.
< $500/month budget	→ Self-hosted on a VM	A single GPU VM beats managed pricing at low scale.
Regulated industry	→ Check managed provider compliance	Azure and AWS have pre-certified HIPAA environments.
High traffic + GPU cost pressure	→ Self-hosted + BentoML/Triton	Managed cloud markup is 3–5× raw compute cost.
Open-source LLM	→ vLLM self-hosted or HF Endpoints	Bedrock and Vertex don't serve arbitrary open models.

TOPIC 05

Release strategies

How do you update the service without breaking the users calling /predict right now?

WHAT YOU JUST BUILT

BentoML service running in Docker · Triton backend handling GPU batching · vLLM endpoint for any LLM · 100 users hitting /predict right now

THE PROBLEM

Model v2 trained — MAE improved 12% · You need to deploy it · Naive approach: stop server, replace model, restart · Result: downtime, and if v2 is broken, users are down

Four release strategies solve this

Each with a different risk, cost and speed tradeoff

01

Blue/Green

Instant switch — zero downtime

02

Canary

Gradual — catch issues early

03

A/B Test

Measure business impact

04

Shadow

Zero risk — test silently

Roll out without breaking users

A new model that passes all tests can still fail in production

01 Blue / Green `Instant switch`

Zero-downtime deploy. Used by Netflix (instant switch between service versions) and most SaaS B2B products.

Run two identical environments. Route 100% to Blue (current). Deploy to Green, test, flip the load balancer.

Instant rollback — flip back to Blue

Costs 2× infra while both environments run

02 Canary Release `Gradual rollout`

Amazon runs hundreds of canary deploys simultaneously. Any model degrading add-to-cart by more than 0.01% is auto-rolled back in 15 minutes.

Route 5% to the new model. Watch MAE and error rate. If healthy → 20% → 50% → 100%. Auto-rollback if metrics degrade.

Real traffic validates gradually

Needs traffic splitting plus monitoring

Measure impact, or take no risk

A/B proves business value; shadow proves safety

03 A/B Testing `Experiment`

Booking.com runs 1000+ A/B tests at once. Ranking model improvements are validated by booking conversion — not NDCG.

Split users by segment. Model A serves group A, B serves group B. Measure CTR, revenue and retention — not MAE.

Proves actual business value

Needs more users and a longer run time

04 Shadow Mode `Zero-risk eval`

Meta uses shadow mode before every major Feed ranking swap — logging shadow vs production for 48h before any traffic shifts.

The new model receives every request in parallel. Predictions are logged but never returned to users. Compare offline.

Absolutely zero risk to users

Doesn't measure user impact. Double compute.

Blue/green — how to implement

Traffic splitting with nginx, Docker labels, or a load balancer config

docker-compose.yml

```yaml
# docker-compose.yml — Blue/Green with nginx
services:
  nginx:
    image: nginx:alpine
    ports: ["8000:80"]
    volumes: ["./nginx.conf:/etc/nginx/nginx.conf"]
    depends_on: [blue, green]

  blue:    # current production model
    image: yourusername/ride-api:v1.2.0

  green:   # new candidate model
    image: yourusername/ride-api:v1.3.0

# nginx.conf — Blue/Green: 100% to blue
upstream backend {
    server blue:8000;   # comment out to switch
    # server green:8000;
}
```

Blue/green switching

Flip the comment in the upstream block, then nginx -s reload. Traffic moves instantly with zero dropped connections.

nginx -s reload

Sends SIGHUP to the master process. Config reloads with zero downtime — in-flight requests complete normally.

Canary — weighted routing, stage by stage

Edit nginx.conf, reload, watch the metrics, advance

nginx.conf

```
# nginx.conf — Canary: weighted routing
upstream backend {
    server blue:8000  weight=95;   # 95% production
    server green:8000 weight=5;    # 5% canary
}

# Canary rollout stages — edit nginx.conf + reload:
# Stage 1:  weight=95 / weight=5    -> watch 30 min
# Stage 2:  weight=80 / weight=20   -> watch 1 hour
# Stage 3:  weight=50 / weight=50   -> watch 2 hours
# Stage 4:  weight=0  / weight=100  -> fully rolled out

# nginx -s reload   # zero-downtime config reload
```

Canary weight=5

Start at 5% — enough real traffic to surface issues, small enough that only 1 in 20 users sees the new model.

When to advance

Advance only if p95 is stable, MAE is no worse than baseline, and the error rate is 0%. Automate this check in CI.

Rollback

At any stage: set the weight back to 100/0 and reload. Done in seconds, no redeployment.

A/B testing — deterministic split by user_id

Same user, same variant, every request

ab_test.py

```python
# A/B Testing — deterministic split by user_id
import hashlib
from fastapi import Request

def get_model_for_user(user_id: str):
    bucket = int(hashlib.md5(
        user_id.encode()).hexdigest(), 16) % 100
    return model_a if bucket < 50 else model_b

@app.post("/predict")
async def predict(req: PredictRequest, request: Request):
    user_id = request.headers.get("X-User-ID", "anonymous")
    model   = get_model_for_user(user_id)
    result  = model.predict([[req.distance_km,
        req.passengers, req.hour_of_day]])
    log_event(user_id=user_id,
        variant="A" if model == model_a else "B",
        prediction=result[0])
    return {"duration_min": round(float(result[0]), 2)}
```

Deterministic A/B

An MD5 hash of user_id modulo 100 gives stable assignment. The same user always sees the same model.

Log everything

Without logging the variant and prediction per user, you can't measure the business impact of B versus A.

Significance

Run for at least 1000 predictions per variant before drawing conclusions. Shorter runs give false signals.

Shadow mode — logged, never returned

Fire and forget: zero added latency on the production path

shadow.py

```python
# Shadow Mode — separate endpoint definition
import asyncio

@app.post("/predict")
async def predict(req: PredictRequest):
    features = [[req.distance_km, req.passengers,
                 req.hour_of_day]]
    prod_result = production_model.predict(
        features)[0]           # user sees this

    async def shadow_eval():
        shadow_result = shadow_model.predict(features)[0]
        log_comparison(prod=prod_result,
            shadow=shadow_result,
            diff=abs(prod_result - shadow_result))

    asyncio.create_task(shadow_eval())   # fire & forget
    return {"duration_min": round(float(prod_result), 2)}
```

asyncio.create_task()

The shadow evaluation runs concurrently without blocking the response. Zero added latency on the production path.

Shadow → Canary → A/B

The recommended order: shadow (zero risk), then canary (real impact), then A/B (business metrics).

Release strategies — comparison

Rollback speed, user risk, infra cost, and what each one actually measures

STRATEGY	ROLLBACK	USER RISK	INFRA COST	BUSINESS IMPACT	BEST FOR
Blue / Green	Instant (flip LB)	Zero during switch	2× while both run	No — technical only	Low-risk deploys, strict rollback SLA
Canary	Fast (re-weight)	< 5% of users	~1× + small canary	Partially (real errors)	Most model updates — best default
A/B Testing	Slow (wait for significance)	50% of users	~2× (two models live)	Yes — conversion, revenue, retention	When model quality ≠ business value
Shadow Mode	N/A (users unaffected)	Zero	~2× (double inference)	No — offline comparison only	Highest-risk changes, new architectures

Quick decision guide

Four questions, four answers

Highest-risk change?

→ Shadow first

Need business metrics?

→ A/B test

Need instant rollback?

→ Blue/green

Everything else?

→ Canary ★

Skip shadow if the model is cheap, low-risk, or the team can't run double compute. Canary is the right default for most ML teams; A/B only when you have enough traffic for statistical significance.

SESSION 3 PROJECT · DELIVERABLES 01–04

From pattern to production

Take the ride-duration model through all three inference patterns

01	**Airflow DAG**	extract → train → evaluate → register pipeline with weekly schedule, retry logic, and dependency chaining.
02	**Web Service**	BentoML with micro-batching running locally via Docker. /predict and /healthz endpoints responding.
03	**Batch Scorer**	Python script reading local Parquet files, scoring with the Production model, writing predictions back.
04	**Event-Driven Consumer**	Broker-agnostic consumer (Kafka/Redis). Predicts on every incoming event. Document latency at 100 events/sec.

SESSION 3 PROJECT · DELIVERABLES 05–08

Serve, measure, roll out

Four deliverables that turn the service into a documented rollout

05	**vLLM Serving**	Serve Qwen2.5-7B-Instruct via vLLM. Call it with the OpenAI SDK. Test streaming. Document TTFT and throughput.
06	**Locust Load Test**	Ramp to 100 concurrent users on the web service. Report p50/p95/p99 latency. Identify and document the bottleneck.
07	**Canary Rollout**	nginx config with a weight=5/95 split between two model versions. Document rollout stages and rollback procedure.
08	**Architecture Diagram**	One diagram showing all three inference patterns sharing the same MLflow Registry. In the README.

Quick reference

BentoML, vLLM, nginx, Docker and Locust

BENTOML + VLLM CLI

`bentoml models list`	List saved models
`bentoml serve service:svc --reload`	Dev with hot-reload
`bentoml build`	Package as a Bento artifact
`bentoml containerize <name:tag>`	Build a Docker image
`bentoml models get <name:tag>`	Show model metadata
`vllm serve <model> --port 8000`	Start a vLLM server
`vllm serve … --tensor-parallel-size 2`	Shard across 2 GPUs
`curl localhost:8000/v1/models`	List vLLM loaded models

NGINX / DOCKER / LOCUST

`nginx -s reload`	Reload config — zero downtime
`nginx -t`	Test config syntax
`docker compose up -d`	Start all services
`docker compose logs -f nginx`	Watch live nginx logs
`docker compose stop green`	Stop the canary for rollback
`docker stats`	Monitor CPU/RAM per container
`locust -f f.py --headless --csv=r`	Headless Locust for CI
`ab -n 1000 -c 50 http://host/predict`	Quick Apache Bench test

RESOURCES

Resources for Session 3

Everything on this slide is free

Airflow Official Docs
airflow.apache.org/docs

NVIDIA Triton
docs.nvidia.com/deeplearning/triton-inference-server

Marc Lamberti — Airflow
youtube.com/@MarcLamberti

Martin Fowler — Canary Release
martinfowler.com/bliki/CanaryRelease.html

Locust Docs
docs.locust.io

MLOps Zoomcamp (free)
github.com/DataTalksClub/mlops-zoomcamp

k6 Docs
k6.io/docs

Made With ML
madewithml.com

BentoML Docs
docs.bentoml.com

TensorFlow Serving overview
tensorflow.org/tfx/guide/serving

vLLM Docs
docs.vllm.ai

Modzy — What is Model Serving
modzy.com/blog/what-is-model-serving

vLLM GitHub
github.com/vllm-project/vllm

NEXT · SESSION 4

Session 4 · Observability and drift detection

Session 3 complete. Push all deliverables to GitHub.

01

Drift Taxonomy

Data drift, concept drift, label drift, embedding drift — why models degrade silently in production.

02

Statistical Detection

PSI, KS test, Chi-square, KL divergence, Page-Hinkley, MMD — when to use each.

03

Evidently AI

Automated data quality and drift reports. Test suites for threshold alerting and CI/CD.

04

Prometheus + Grafana

Instrument your serving layer. Build a live model monitoring dashboard with drift alerts.

05

Langfuse + RAGAS

LLM observability: traces, spans, faithfulness, answer relevancy, context recall, cost monitoring.

Questions?

Session 3 complete. Push all deliverables to GitHub.

Aya · MLOps MENA Community · ITI

https://www.linkedin.com/company/mlops-mena #mlops-mena